UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science
Faculty of Computing

Optimizing High-Performance Data Processing for Large-Scale Web Crawlers

Programme	:	Bachelor of Computer Science *(Data Engineering)*
Subject Code	:	SECP3133
Subject Name	:	High-Performance Data Processing
Session-Sem	:	2025/2026-2

Prepared by	:	LAU YAN KAI	(A23CS0098)
		CHEW CHIU XIAN	(A23CS0061)
		ELIJAH SHE YU SHENG	(A23CS0073)
		NEO LI XIN	(A23CS0253)
Section	:	02	
Lecturer	:	DR. SEAH CHOON SEN	
Date	:	26 June 2026	

Table of Content

1.0 Introduction

1.1 Background of the project

The rapid growth of the internet has resulted in an enormous amount of publicly available data being generated every day. Organizations increasingly rely on large-scale data collection and processing to support decision-making, market analysis, and business intelligence. Web crawling and web scraping have become essential techniques for gathering structured information from online sources efficiently.

However, collecting and processing large volumes of web data can be time-consuming when performed using traditional sequential approaches. As datasets grow in size, high-performance computing (HPC) techniques become important for improving processing speed, scalability, and resource utilization. Techniques such as multithreading, multiprocessing, and parallel data processing enable systems to handle large workloads more efficiently.

This project focuses on the development of a large-scale web crawler capable of collecting more than 100,000 property records from iProperty Malaysia. The collected data will undergo cleaning, transformation, and optimization processes before being analyzed. High-performance computing techniques will be applied to improve the efficiency of the crawling and data processing pipeline, followed by a performance evaluation to measure the impact of the implemented optimizations.

1.1.1 Web Scraping

Web crawling refers to the automated process of navigating through web pages and discovering relevant content by following hyperlinks and pagination structures. Web scraping is the process of extracting specific information from the discovered web pages and converting it into a structured format suitable for analysis.

In this project, a web crawler will be developed to navigate through property listing pages on iProperty Malaysia. The scraper component will then extract relevant information

such as property title, price, location, number of bedrooms, property type, and other available attributes. The extracted information will be stored in a structured dataset for further processing and analysis.

1.1.2 Data Processing

Raw data collected from websites often contains inconsistencies, duplicates, missing values, and formatting issues. Data processing is necessary to improve data quality and ensure that the dataset is suitable for analysis.

The data processing stage of this project includes data cleaning, validation, transformation, and formatting. Duplicate records will be removed, missing values will be handled appropriately, and selected attributes will be standardized into consistent formats. These processes help improve the reliability and usability of the final dataset.

1.1.3 Optimisation Process

As the number of records increases, the computational resources required for data collection and processing also grow significantly. Sequential execution may lead to longer processing times and inefficient resource utilization.

To address these challenges, high-performance computing techniques will be implemented to optimize the web crawling and data processing pipeline. Techniques such as multithreading and multiprocessing will be used to execute tasks concurrently, reducing overall execution time while improving throughput. The effectiveness of these optimization techniques will be evaluated through performance metrics including execution time, CPU utilization, memory consumption, and throughput.

1.2 Objectives

The objectives of this project are:

1. To develop a web crawler capable of collecting at least 100,000 property records from iProperty Malaysia.
2. To extract and store property information in a structured format suitable for analysis.
3. To perform data cleaning, transformation, and validation on the collected dataset.
4. To implement high-performance computing techniques such as multithreading and multiprocessing to optimize the crawling and processing pipeline.
5. To evaluate and compare the performance of the baseline and optimized implementations using quantitative metrics.

1.3 Target Website and Data to be Extracted

The selected target website for this project is iProperty Malaysia, one of the largest online property marketplaces in Malaysia. The platform contains a large number of property listings across different states and property categories, making it suitable for large-scale data collection. At the time of verification, the website contained more than 212,000 property listings for sale, exceeding the minimum project requirement of 100,000 records.

The crawler will extract structured information from property listings, including:

- Property title
- Property price
- Property location
- Property type
- Number of bedrooms
- Number of bathrooms
- Built-up area
- Listing category
- Property description (if available)

The collected data will be stored in CSV format and used throughout the data processing and performance evaluation stages of the project.

2.0 System Design & Architecture

2.1 System Architecture

The proposed system in Figure 2.1 follows a multi-stage data processing pipeline consisting of data collection, data processing, optimization, and performance evaluation components.

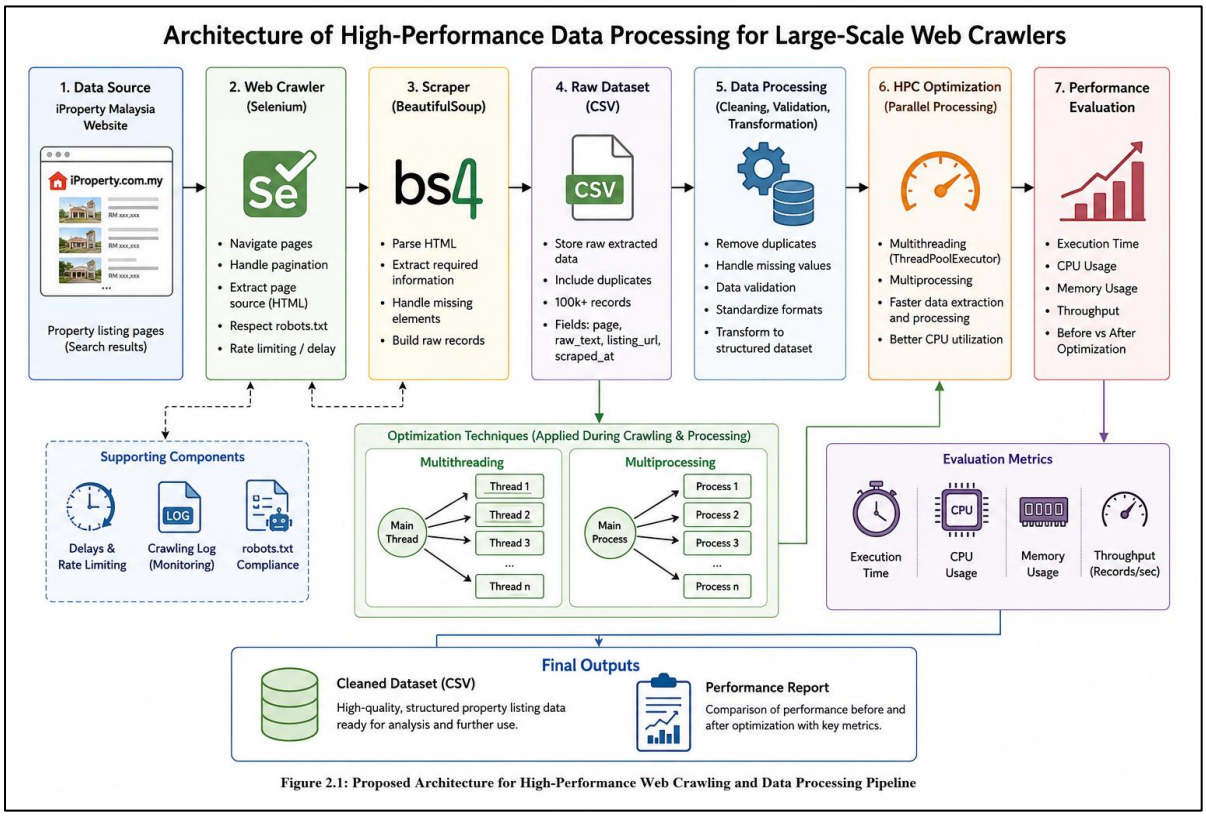


Figure 2.1 Proposed Architecture for High-Performance Web Crawling and Data Processing Pipeline

The process begins with the web crawler, which sends requests to iProperty Malaysia and navigates through listing pages. Property URLs are collected and passed to the scraper module, where relevant property information is extracted from each listing page.

The extracted data is then stored in a raw dataset. During the data processing stage, the dataset undergoes cleaning, validation, and transformation to improve data quality and consistency. The cleaned dataset is subsequently used for performance optimization experiments.

Optimization techniques such as multithreading and multiprocessing are applied to improve the efficiency of the crawling and processing tasks. Performance metrics including execution time, CPU utilization, memory usage, and throughput are collected and analyzed to evaluate the effectiveness of the optimizations.

The final output consists of a cleaned property dataset and a performance comparison report demonstrating the impact of the implemented HPC techniques.

2.2 Tools and Frameworks Used

Table 2.1 shows the following tools and frameworks are utilized in this project.

Table 2.1 Tools and Frameworks Used in the Project

Component	**Tools / Frameworks Used**
Programming Language	Python
Web Crawling	Selenium, ChromeDriver, webdriver-manager, BeautifulSoup
Data Processing	Pandas
Optimization	Requests, BeautifulSoup, ThreadPoolExecutor
Data Storage	CSV File
Performance Monitoring	Pandas, Crawling Log, CMD Output
Development Environment	Windows CMD, VS Code / Notepad, Python Virtual Environment

2.3 Roles of Team embers

The project tasks are divided among team members to ensure efficient collaboration and workload distribution as shown in Table 2.2.

Table 2.2 Roles and Responsibilities of Each Team Member

Member	Role	Responsibilities
Neo Li Xin	Project Lead and System Architect	Project planning, target dataset selection, architecture design and GitHub repository management
Elijah She Yu Sheng	Data Collection Engineer	Web crawler development, data extraction, dataset generation
Lau Yan Kai	Data Processing Engineer	Data cleaning, transformation, validation, storage
Chew Chiu Xian	HPC and Performance Specialist	Optimization implementation, benchmarking, performance evaluation, visualization

3.0 Data Collection

The data collection phase focused on collecting large-scale property listing data from iProperty Malaysia. The target website selected for this project was the iProperty Malaysia property-for-sale page, which provides publicly available property listing information across multiple pages. This website was suitable for the project because it contains a large number of property listings and provides useful property-related information such as property price, location, floor size, number of bedrooms, number of bathrooms, property type, listing URL, and listing timestamp.

The collected data was saved as a raw CSV dataset before further data cleaning and processing. At this stage, the purpose was to preserve the original extracted listing information without modifying the values. Therefore, the raw dataset contains the source page number, raw listing text, listing URL, and scraping timestamp. This raw format allows the data processing team to later extract structured fields such as price, location, bedrooms, bathrooms, floor size, and property type during the cleaning phase.

3.1 Crawling Method

The web crawler was developed using Python with Selenium and BeautifulSoup. Selenium was used to automate browser navigation and access the paginated iProperty property-for-sale pages, while BeautifulSoup was used to parse the HTML content and extract property listing information from the loaded page source.

The crawler accessed the website page by page using the pagination parameter in the URL. For each page, the crawler opened the page in Chrome, waited for the page body to load, extracted all relevant listing links and listing text, and saved the extracted records into a CSV file. Each record was stored with the page number, raw listing text, listing URL, and the timestamp of extraction.

The general crawling workflow is as follows:

1. Open the iProperty Malaysia property-for-sale page.
2. Navigate through listing pages using the page number parameter.
3. Wait for the webpage content to load using Selenium.
4. Extract the page source after loading.
5. Parse the HTML content using BeautifulSoup.
6. Identify property listing records based on listing text and listing URL.
7. Save the extracted records into a raw CSV file.
8. Log the crawling status for each page.
9. Continue the process until more than 100,000 records are collected.

During development, a faster Requests-based crawler with multithreading was also tested. Although it successfully collected data from the early pages, it later returned HTTP 403 access forbidden errors during larger-scale crawling. Therefore, Selenium was selected as the final crawling method because it provided more stable browser-based access for long-duration data collection.

To reduce the risk of overloading the website, random delays were added between page requests. The crawler also recorded crawling logs, including the page number, status, number of records collected, timestamp, and error message if a page failed. This made it easier to track crawling progress and resume the process from the last successful page when needed.

3.2 Number of records collected

The final raw dataset contains 102,502 property listing records collected from iProperty Malaysia. The data was collected from multiple paginated property-for-sale pages, with the crawling process continuing until the dataset exceeded the minimum requirement of 100,000 records.

The raw dataset contains four main column as shown in Table 3.1.

Table 3.1 Column Definition

Column Name	**Description**
page	The page number where the listing was collected
raw_text	The original extracted text from the property listing card
listing_url	The URL link of the property listing
scraped_at	The date and time when the record was collected

The number of records collected is summarized as follows in Table 3.2.

Table 3.2 Total Number of Records Collected

Description	**Value**
Total raw records collected	102,502
Unique listing URLs	95,004
Duplicate records	7,498

Description	Value
Dataset format	CSV
Raw dataset file	iproperty_raw_100k_final.csv

Some duplicate records were found because certain property listings appeared more than once across different result pages. Since this phase focuses on raw data collection, the duplicate records were retained in the raw dataset for transparency. These duplicate records will be handled later during the data cleaning stage.

Figure 3.1 shows the command prompt output after completing the web crawling process. The output confirms that the crawler successfully collected more than 100,000 raw property listing records from iProperty Malaysia. This screenshot serves as evidence that the data collection process achieved the minimum dataset size required for the project.

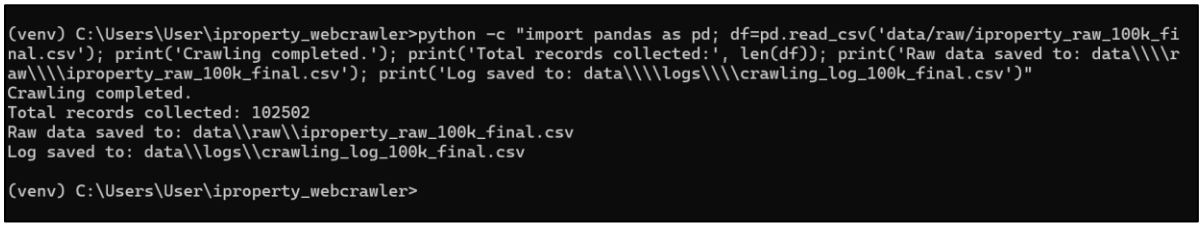
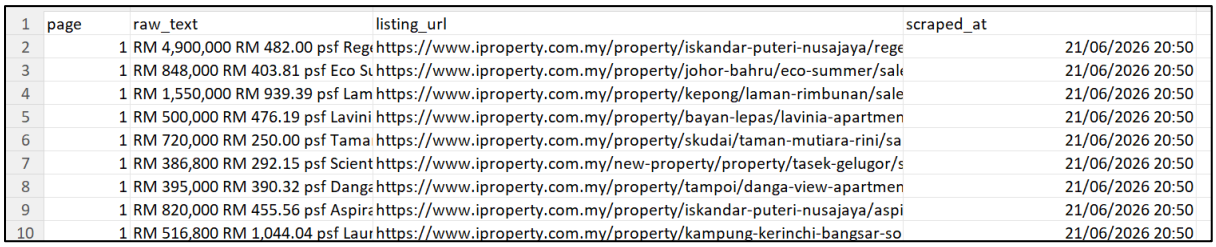


Figure 3.1 CMD Output Showing Total Records Collected

Figure 3.2 presents the summary statistics of the final raw dataset using Python Pandas. The dataset contains 102,502 rows and 4 columns, which confirms that the raw dataset exceeds the 100,000-record requirement. The output also shows 95,004 unique listing URLs and 7,498 duplicate records. These duplicate records were retained in the raw dataset because this stage focuses on raw data collection, while duplicate removal will be performed during the data cleaning stage.

Figure 3.2 Dataset Shape, Unique Links and Duplicate Records

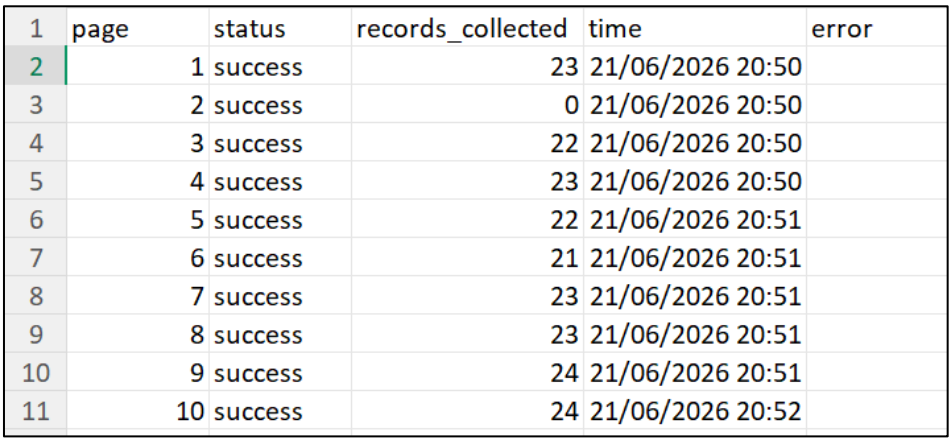
Figure 3.3 shows a preview of the final raw dataset, iproperty_raw_100k_final.csv, opened in Excel. The dataset contains four main fields: page, raw_text, listing_url, and scraped_at. The raw_text column stores the original extracted listing information, while the listing_url column stores the property listing link. This confirms that the crawler saved the collected property data in a structured CSV format for the next processing stage.



	page	raw_text	listing_url	scraped_at
2	1	RM 4,900,000 RM 482.00 psf Rege	https://www.iproperty.com.my/property/iskandar-puteri-nusajaya/rege	21/06/2026 20:50
3	1	RM 848,000 RM 403.81 psf Eco Su	https://www.iproperty.com.my/property/johor-bahru/eco-summer/sale	21/06/2026 20:50
4	1	RM 1,550,000 RM 939.39 psf Lam	https://www.iproperty.com.my/property/kepong/laman-rimbunan/sale	21/06/2026 20:50
5	1	RM 500,000 RM 476.19 psf Lavini	https://www.iproperty.com.my/property/bayan-lepas/lavinia-apartmen	21/06/2026 20:50
6	1	RM 720,000 RM 250.00 psf Tama	https://www.iproperty.com.my/property/skudai/taman-mutiara-rini/sa	21/06/2026 20:50
7	1	RM 386,800 RM 292.15 psf Scient	https://www.iproperty.com.my/new-property/property/tasek-gelugor/s	21/06/2026 20:50
8	1	RM 395,000 RM 390.32 psf Danga	https://www.iproperty.com.my/property/tampoi/danga-view-apartmen	21/06/2026 20:50
9	1	RM 820,000 RM 455.56 psf Aspira	https://www.iproperty.com.my/property/iskandar-puteri-nusajaya/aspi	21/06/2026 20:50
10	1	RM 516,800 RM 1,044.04 psf Laur	https://www.iproperty.com.my/property/kampung-kerinchi-bangsar-so	21/06/2026 20:50

Figure 3.3 Preview of Final Raw Dataset in CSV Format

Figure 3.4 shows the crawling log generated during the data collection process. The log records the page number, crawling status, number of records collected, timestamp, and error message if any issue occurred. This log was useful for monitoring crawler progress, identifying failed pages, and resuming the crawling process from the last successful page when needed. It also improves the reliability and traceability of the data collection process.



	page	status	records_collected	time	error
2	1	success	23	21/06/2026 20:50	
3	2	success	0	21/06/2026 20:50	
4	3	success	22	21/06/2026 20:50	
5	4	success	23	21/06/2026 20:50	
6	5	success	22	21/06/2026 20:51	
7	6	success	21	21/06/2026 20:51	
8	7	success	23	21/06/2026 20:51	
9	8	success	23	21/06/2026 20:51	
10	9	success	24	21/06/2026 20:51	
11	10	success	24	21/06/2026 20:52	

Figure 3.4 Crawling Log File Preview

3.3 Ethical Considerations

The data collection process was conducted for academic purposes only. The crawler collected publicly available property listing information from iProperty Malaysia and did not access private, login-protected, or sensitive user data. The collected information was limited to

property listing details that were already visible on the website, such as listing text, property URL, and listing-related attributes.

Several ethical considerations were applied during the crawling process. First, the crawler used random delays between page requests to reduce server load and avoid sending excessive requests in a short period of time. Second, the crawler did not attempt to bypass login systems, payment restrictions, CAPTCHA mechanisms, or security protections. Third, the crawler maintained a crawling log so that failed pages could be tracked and the process could be resumed responsibly without repeatedly sending unnecessary requests to the same pages.

In addition, a Requests-based multithreaded crawler was tested but was not used as the final method after the website returned HTTP 403 access forbidden responses. This decision was made to avoid aggressive request behaviour and to maintain a more responsible crawling approach. Therefore, the final crawler used Selenium with controlled page-by-page crawling and delay intervals to collect the required dataset in a more stable and ethical manner

4.0 Data Processing

The data processing stage is a crucial link that connects the collection of raw web data with the high-performance computing optimization in the pipeline. The text from the Malaysian real estate portal iProperty is inherently unstructured, containing multiple lines of text fragments, inconsistent HTML layout remnants, and variable data formats.

The main objective of this module is to process, clean, and transform the unprocessed web text into a clearly structured and fully verified table format. As the project has strict requirements for at least 100,000 valid records, the design of the data processing flow focuses on ensuring data retention and data integrity, while also guaranteeing the accuracy of downstream analysis.

4.1 Cleaning Methods

To ensure that the high-performance data pipeline can maintain data integrity completely, we have implemented a strict data cleaning process to achieve the main goals of eliminating duplicates, standardizing field formats, handling missing values, and verifying data types.

As shown in Figure 4.1, the initial stage of the cleaning process focuses on rigorous deduplication operations, as large-scale web crawlers often encounter duplicate entries due to overlapping pagination or server-side caching anomalies. By setting the unique listing_url field as the primary identifier, the script uses non-destructive filtering technology to systematically delete all duplicate records. This method retains the characteristic of each record having a unique chronological order from the beginning, thus ensuring data integrity while maintaining a clear baseline.

```
# Remove duplicate
initial_count = len(df)
df.drop_duplicates(subset=['listing_url'], keep='first', inplace=True)
print(f"Removed {initial_count - len(df)} duplicate rows.")

Removed 7498 duplicate rows.
```

Figure 4.1 Remove Duplicate

As shown in Figure 4.2, to establish a predictable schema layout before processing the raw web logs, the data processing pipeline begins by pre-allocating memory and initializing the target destination columns within the DataFrame to prevents runtime exceptions during localized cell assignments, ensures downstream optimization scripts recognize the correct data schemas, and avoids data fragmentation warnings.

```
# Initialize new columns to store extracted data
df['price_rm'] = np.nan
df['price_psf'] = np.nan
df['built_up_sqft'] = np.nan
df['land_area_sqft'] = np.nan
df['property_type'] = ""
df['furnishing'] = ""
```

Figure 4.2 Structural Schema Initialization

Figure 4.3 shows that once the destination schema is initialized, the pipeline executes an iterative extraction engine utilizing a sequential loop combined with the df.iterrows() generator to parse the unstructured raw text row by row. For each entry, the string representation of the crawled web log is passed through a sequence of deterministic Regular Expressions configured to isolate numeric tokens from messy text strings. The patterns isolate financial and spatial attributes such as baseline asset prices, price per square foot metrics, and indoor built-up or outdoor land dimensions while stripping away currency tags, punctuation, commas, and formatting strings before casting the final filtered character strings directly into machine-readable float64 data types.

```python
for index, row in df.iterrows():
  text = str(row['raw_text'])

  # 1. Extract Total Price (Grabs the starting price if it's a range)
  price_match = re.search(r'RM\s*([\d,]+)', text)
  if price_match:
    df.at[index, 'price_rm'] = float(price_match.group(1).replace(',', ''))

  # 2. Extract Price Per Sqft
  psf_match = re.search(r'RM\s*([\d,.]+)\s*psf', text, flags=re.IGNORECASE)
  if psf_match:
    df.at[index, 'price_psf'] = float(psf_match.group(1).replace(',', ''))

  # 3. Extract Built-up / Floor Size
  floor_match = re.search(r'([\d,]+)\s*sqft(?:\s*\(floor\))?', text, flags=re.IGNORECASE)
  if floor_match:
    df.at[index, 'built_up_sqft'] = float(floor_match.group(1).replace(',', ''))

  # 4. Extract Land Size
  land_match = re.search(r'([\d,]+)\s*sqft\s*\(land\)', text, flags=re.IGNORECASE)
  if land_match:
    df.at[index, 'land_area_sqft'] = float(land_match.group(1).replace(',', ''))

  # 5. Extract Property Type
  if 'Bungalow' in text or 'Villa' in text:
    df.at[index, 'property_type'] = 'Bungalow/Villa'
  elif 'Condominium' in text or 'Serviced Residence' in text:
    df.at[index, 'property_type'] = 'Condominium'
  elif 'Apartment' in text or 'Flat' in text:
    df.at[index, 'property_type'] = 'Apartment/Flat'
  elif 'Terrace' in text or 'Terraced' in text or 'Link' in text:
    df.at[index, 'property_type'] = 'Terrace/Link'
  elif 'Semi-D' in text or 'Semi-Detached' in text:
    df.at[index, 'property_type'] = 'Semi-Detached'

  # 6. Extract Furnishing Status
  if 'Unfurnished' in text:
    df.at[index, 'furnishing'] = 'Unfurnished'
  elif 'Partly' in text or 'Partially' in text:
    df.at[index, 'furnishing'] = 'Partially Furnished'
  elif 'Fully' in text:
    df.at[index, 'furnishing'] = 'Fully Furnished'
```

Figure 4.3 Regular Expression and Categorical Extraction

Figure 4.4 shows the second stage of the methodology addresses the crucial issue of missing value imputation. This stage requires careful handling to ensure that the dataset contains at least 100,000 valid records, meeting the project's strict requirements for data volume. To avoid randomly deleting rows with missing attributes and thereby degrading the quality of the dataset below the project's set standards, a mixed mathematical approach was adopted to ensure the integrity of the sample size. For missing values in numerical fields such as price and building area, the system calculates and inputs the median of each column to prevent structural distortions caused by extreme luxurious properties when compared to the average value. On the other hand, for blank items in categorical fields such as missing furniture information or unclassified property details, a standardized string placeholder “Unknown” is assigned, maintaining the consistency of the data structure throughout the dataset.

```python
# Fill missing values with the median
df['price_rm'] = df['price_rm'].fillna(df['price_rm'].median())
df['price_psf'] = df['price_psf'].fillna(df['price_psf'].median())

# Fill missing categorical values with 'Unknown'
df['property_type'] = df['property_type'].replace("", "Unknown")
df['furnishing'] = df['furnishing'].replace("", "Unknown")

# Convert 'scraped_at' to DateTime object
df['scraped_at'] = df['scraped_at'].str.replace(r'\s+', ' ', regex=True).str.strip()
df['scraped_at'] = pd.to_datetime(df['scraped_at'], dayfirst=True, format='mixed', errors='coerce')
```

Figure 4.4 Data Cleaning

Lastly, as shown in the Figure 4.5, the complete cleaning system was designed by reassigning the design based on clear data, to comply with modern structured programming standards and strict type verification requirements. The code intentionally avoids using the outdated inplace=True parameter on independent column slices, thereby completely eliminating the risks brought by chained assignments and protecting the data flow from internal memory warnings. This deliberate architectural design ensures that all standardized fields, structure formats, and data types are in a neat and consistent state, forming a data set that can be directly put into production and is highly suitable for high-performance distributed computing systems.

```python
final_columns = [
        'listing_url',
        'property_type',
        'price_rm',
        'price_psf',
        'built_up_sqft',
        'land_area_sqft',
        'furnishing',
        'scraped_at'
    ]

cleaned_df = df[final_columns]

cleaned_df = cleaned_df.astype({
        'price_rm': 'float64',
        'price_psf': 'float64',
        'property_type': 'string',
        'furnishing': 'string'
    })

output_filepath = '/content/iproperty_cleaned.csv'
```

```python
# Export cleaned dataframe
cleaned_df.to_csv(output_filepath, index=False)
print(f"Dataset successfully saved to {output_filepath}")
```

```
Dataset successfully saved to /content/iproperty_cleaned.csv
```

```python
display(cleaned_df.head(10))
```

	listing_url	property_type	price_rm	price_psf	built_up_sqft	land_area_sqft	furnishing	scraped_at
0	https://www.iproperty.com.my/property/iskandar...	Bungalow/Villa	4900000.0	482.00	7200.0	10166.0	Unfurnished	2026-06-21 20:50:17
1	https://www.iproperty.com.my/property/johor-ba...	Terrace/Link	848000.0	403.81	1600.0	2100.0	Unfurnished	2026-06-21 20:50:17
2	https://www.iproperty.com.my/property/kepong/l...	Terrace/Link	1550000.0	939.39	3600.0	1650.0	Fully Furnished	2026-06-21 20:50:17
3	https://www.iproperty.com.my/property/bayan-le...	Apartment/Flat	500000.0	476.19	1050.0	NaN	Partially Furnished	2026-06-21 20:50:17
4	https://www.iproperty.com.my/property/skudai/t...	Terrace/Link	720000.0	250.00	1980.0	2880.0	Partially Furnished	2026-06-21 20:50:17
5	https://www.iproperty.com.my/new-property/prop...	Terrace/Link	386800.0	292.15	1324.0	NaN	Unknown	2026-06-21 20:50:17
6	https://www.iproperty.com.my/property/tampoi/d...	Apartment/Flat	395000.0	390.32	1012.0	NaN	Unfurnished	2026-06-21 20:50:17
7	https://www.iproperty.com.my/property/iskandar...	Terrace/Link	820000.0	455.56	2238.0	1800.0	Unfurnished	2026-06-21 20:50:17
8	https://www.iproperty.com.my/property/kampung-...	Unknown	516800.0	1044.04	495.0	NaN	Partially Furnished	2026-06-21 20:50:17
9	https://www.iproperty.com.my/new-property/prop...	Terrace/Link	1010000.0	397.32	2542.0	NaN	Unknown	2026-06-21 20:50:17

Figure 4.5 Exporting Data to New File

4.2 Data Structure

The processed data structure transforms the pipeline from random string fragments into a standard relational model saved in an organized CSV format. Table 1 shows the final data structure implements strict schema validation and type restrictions.

Field	Data Type	Structural Purpose
listing_url	string	Unique resource identifier; acts as the primary key
property_type	string	Normalized property classification (Condominium, Terrace/Link)
price_rm	float64	Total asset cost in Malaysian Ringgit (MYR)
price_psf	float64	Calculated valuation metric (Price per square foot)
built_up_sqft	float64	Total indoor usable floor area
land_area_sqft	float64	Total geometric footprint size, otherwise NaN
furnishing	string	Asset status categorization (Unfurnished, Partially, Fully)
scraped_at	Date Time	Standardized temporal stamp of pipeline ingestion

After the conversion loop, immediately use .astype() to achieve strict type conversion. This ensures that memory allocation is optimized, thereby avoiding pattern mismatch errors when processing data blocks in subsequent distributed optimization frameworks such as Apache Spark or Dask.

4.3 Transformation and Formatting

Since the unoptimized benchmark crawler generates a merged text string for each list, structural analysis is crucial for converting the collected data into the standardized data model specified by the course requirements. Figure 4.6 shows that this process utilizes the built-in string processing functions and regular expressions in Python to identify specific prefix and suffix anchors in the text. This systematic extraction method can transform the complex nested front-end layout into independent, self-contained values, providing the team with a structured foundation that enables them to meet the technical requirements of the project for generating a fully verified dataset.

```
for index, row in df.iterrows():
  text = str(row['raw_text'])

  # 1. Extract Total Price (Grabs the starting price if it's a range)
  price_match = re.search(r'RM\s*([\d,]+)', text)
  if price_match:
    df.at[index, 'price_rm'] = float(price_match.group(1).replace(',', ''))

  # 2. Extract Price Per Sqft
  psf_match = re.search(r'RM\s*([\d,.]+)\s*psf', text, flags=re.IGNORECASE)
  if psf_match:
    df.at[index, 'price_psf'] = float(psf_match.group(1).replace(',', ''))

  # 3. Extract Built-up / Floor Size
  floor_match = re.search(r'([\d,]+)\s*sqft(?:\s*\(floor\))?', text, flags=re.IGNORECASE)
  if floor_match:
    df.at[index, 'built_up_sqft'] = float(floor_match.group(1).replace(',', ''))

  # 4. Extract Land Size
  land_match = re.search(r'([\d,]+)\s*sqft\s*\(land\)', text, flags=re.IGNORECASE)
  if land_match:
    df.at[index, 'land_area_sqft'] = float(land_match.group(1).replace(',', ''))


        # 5. Extract Property Type
        if 'Bungalow' in text or 'Villa' in text:
          df.at[index, 'property_type'] = 'Bungalow/Villa'
        elif 'Condominium' in text or 'Serviced Residence' in text:
          df.at[index, 'property_type'] = 'Condominium'
        elif 'Apartment' in text or 'Flat' in text:
          df.at[index, 'property_type'] = 'Apartment/Flat'
        elif 'Terrace' in text or 'Terraced' in text or 'Link' in text:
          df.at[index, 'property_type'] = 'Terrace/Link'
        elif 'Semi-D' in text or 'Semi-Detached' in text:
          df.at[index, 'property_type'] = 'Semi-Detached'

        # 6. Extract Furnishing Status
        if 'Unfurnished' in text:
          df.at[index, 'furnishing'] = 'Unfurnished'
        elif 'Partly' in text or 'Partially' in text:
          df.at[index, 'furnishing'] = 'Partially Furnished'
        elif 'Fully' in text:
          df.at[index, 'furnishing'] = 'Fully Furnished'
```

Figure 4.6 Data Extraction from Raw Text

During this transitional phase, an important technical challenge is the alignment of numerical ranges and the consistency of character case formats across different property listings. For instance, new real estate projects often provide vague price or area limitations rather than precise integers. To address this issue, the extraction expressions are meticulously designed to analyse the text sequence from left to right, capture the first matching block, and

systematically establish a reliable baseline for determining the minimum starting values for asset costs and area measurement. Additionally, by introducing flags such as case-insensitive compilation, this processing flow can easily unify word variants like lowercase and uppercase area indicators without causing errors. At the same time, complex structure chains accept Boolean substring evaluations, converting highly variable manually created real estate descriptions into coherent and standardized classification results, ensuring complete consistency among all entries.

The final step of the conversion process emphasizes the strict standardization of the time text fields, which often contain hidden web characters and inconsistent time formats. During the conversion stage, blank characters are first compressed using regular expressions to eliminate irregular double spaces or non-line HTML spacing elements that may cause errors in the standard date parser. After this pre-processing step, the system will initiate a flexible mixed format date-time conversion, using a clear “day-hour” sequence setting. This enables the system to accurately identify 12-hour (AM/PM) and 24-hour military time formats line by line and automatically convert the strings into clear and unified timestamps. This powerful time adjustment prevents null values from being generated in the target set, ensures the accuracy of time identifiers, and provides excellent scalability for future high-performance computing systems.

5.0 Optimization Techniques

To maximize data throughput and break through the limitations of traditional single-threaded computing, this project has implemented two independent high-performance data processing frameworks. We do not view optimization as a single aspect; instead, we address performance bottlenecks through both the hardware scheduling layer, which are macro-parallelism and the instruction execution layer, which is micro-quantization.

To scientifically evaluate the performance improvements of these engineered architectures, their hardware mechanisms and instruction execution strategies were compared with a traditional, unoptimized single-threaded serial benchmark model. This benchmark model served as a control and was used for our dataset, which included 102,502 uncleaned iProperty real estate listings.

5.1 Methods Used: Technical Analysis of the 2 Optimization Frameworks

The Baseline Model: Single-Threaded Sequential Core Iteration

This framework represents the default, unoptimized data engineering approach. It directly reads the raw text data rows into the host memory and performs element-wise iterative processing through the high-level Python interpreter structure such as.iterrows() or manual item tracking loops. Its operation mechanism entirely relies on sequential processing, with each data row being processed sequentially in order. No parallelization or optimization strategies are employed.

The impact of hardware and bottlenecks is highly significant in this model. Execution is still strictly limited by the global interpreter lock (GIL) of Python and the single-core frequency limit. Although one computing thread is responsible for compiling regular expression patterns and reallocating string memory, all other logical cores are completely idle. This results in a low overall hardware utilization and introduces strict linear execution delays. O(N). As the data volume increases, the processing delay at the O(N) level will significantly rise, making this method increasingly impractical for large-scale data engineering tasks.

Optimized Technique 1: Macro-Parallelism via Process-Based Multiprocessing (MIMD)

To bypass the single-core limitation of GIL, this architecture using concurrent.futures.ProcessPoolExecutor to implement process-based multi-core parallel distribution. The raw dataset matrix is divided horizontally into independent, non-overlapping shards across system memory using NumPy array partitioning layouts:

$$\text{Data Shards}=\text{np.array\_split}(\text{DataFrame},\text{Workers } N)$$

This change in operation mode enables tasks to truly execute in parallel, distributing the workload to multiple independent processes rather than threads. This method has significant optimization effects and advantages in terms of hardware. Each individual segment is directly mapped to an independent sub-process, with its own private virtual memory space and dedicated Python interpreter instance. This reconfigures the execution environment to the MIMD (Multiple Instruction, Multiple Data) state, where physical processor cores can run simultaneously, thereby parallelizing the text parsing task into multiple work threads, and completely avoiding lock contention issues. However, since the system still relies on internal row-by-row iterative loops, which is for loops within each sub-work thread, a single task is still

limited by the overhead of high-level interpreter instructions, which hinders a comprehensive optimization of the hardware.

Optimized Technique 2: Hybrid Framework (Macro-Multiprocessing + Micro-Vectorized Array Masking)

This represents our advanced high-performance data architecture. Although optimization technique 1 has solved the core distribution problem at the hardware level, the child processes may still be affected by the delay in the underlying row iteration. The hybrid framework addresses this issue by combining macro-level sharding processing with micro-level vectorized batch processing, and is driven by the pre-compiled C extension of Pandas. Its operation mechanism strategically combines the two parallel paradigms, thereby eliminating the bottlenecks at each level of the execution pipeline.

The optimization and advantages at the hardware level are achieved through this two-layer architecture, achieving the highest efficiency. In each independent sub-process workspace, manual row-by-row looping is strictly prohibited. Instead, through the underlying pre-compiled C language kernel, the .str.contains() method is used to perform parallel evaluation of the entire array matrix for categorical list attributes such as property models and furniture configurations. Since the regular expression extraction operation is directly executed in the continuous, memory-mapped column blocks, this pipeline fully utilizes the hardware-level SIMD (Single Instruction Multiple Data) processing path. This effectively reduces the interpreter call stack level, eliminates the single-core clock bottleneck, and maximizes the hardware throughput, thereby providing a performance-optimal and scalable solution for large-scale data processing tasks.

5.2 Code Overview or Pseudocode of Techniques Applied

Figure 5.1 provides a detailed illustration of the underlying computational sequence that limits the performance characteristics of the control benchmark. The fundamental reason for the linear scaling penalty O(N) stems directly from the use of the for idx, row in df_clean.iterrows(): iteration structure. At the interpreter level, .iterrows() is inherently inefficient for large-scale data engineering tasks: in each individual loop iteration, the engine

must look up the memory pointer, determine the series index, and encapsulate the underlying raw row scalar into a completely new and more costly Pandas Series object.

```python
def run_sequential_baseline_engine(df_raw):
    print("[Control Baseline] Initializing Single-Threaded Sequential Loop...")
    df_clean = df_raw.copy()
    prop_types, furnishing_list, prices_rm, prices_psf, built_up, land_area = [], [], [], [], [], []

    # Inefficient element-by-element string translation loop
    for idx, row in df_clean.iterrows():
        text = str(row['raw_text'])

        # Categorical String Parsing
        if re.search(r"Bungalow|Villa", text, re.IGNORECASE): prop_types.append("Bungalow/Villa")
        elif re.search(r"Condominium|Serviced Residence", text, re.IGNORECASE): prop_types.append("Condominium")
        elif re.search(r"Apartment|Flat", text, re.IGNORECASE): prop_types.append("Apartment/Flat")
        elif re.search(r"Terrace|Terraced|Link", text, re.IGNORECASE): prop_types.append("Terrace/Link")
        else: prop_types.append("Other")

        # Furnishing Layout Extractions
        if re.search(r"Unfurnished", text, re.IGNORECASE): furnishing_list.append("Unfurnished")
        elif re.search(r"Partly|Partially", text, re.IGNORECASE): furnishing_list.append("Partially Furnished")
        elif re.search(r"Fully", text, re.IGNORECASE): furnishing_list.append("Fully Furnished")
        else: furnishing_list.append("Unknown")

        # Regex Numerical Extraction
        p_rm = re.search(r"RM\s*([\d,]+)", text)
        prices_rm.append(float(p_rm.group(1).replace(",", "")) if p_rm else np.nan)
        p_psf = re.search(r"RM\s*([\d,.]+)\s*psf", text, re.IGNORECASE)
        prices_psf.append(float(p_psf.group(1).replace(",", "")) if p_psf else np.nan)
        b_up = re.search(r"([\d,]+)\s*sqft(?:\s*\(floor\))?", text, re.IGNORECASE)
        built_up.append(float(b_up.group(1).replace(",", "")) if b_up else np.nan)
        l_area = re.search(r"([\d,]+)\s*sqft\s*\(land\)", text, re.IGNORECASE)
        land_area.append(float(l_area.group(1).replace(",", "")) if l_area else np.nan)

    df_clean['property_type'] = prop_types
    df_clean['furnishing'] = furnishing_list
    df_clean['price_rm'] = prices_rm
    df_clean['price_psf'] = prices_psf
    df_clean['built_up_sqft'] = built_up
    df_clean['land_area_sqft'] = land_area
    return df_clean
```

Figure 5.1 Execution Logic Map of Control Baseline Processing Flow

When the program executes the compiled regular expression re.search(), the CPU threads are forced to repeatedly jump between the high-level Python object structure and the underlying precompiled C library. Since this pipeline strictly runs within a single execution path, the Python global interpreter lock (GIL) locks the interpreter's execution state. This prevents any auxiliary processor registers or sibling threads from assisting in extracting the workload, resulting in the multi-core hardware infrastructure being at most 75% idle, while string memory reallocation continuously accumulates processing delays.

Figure 5.2 outlines the process-level data distribution mechanism implemented to overcome the single-core limitation of the interpreter lock. The architecture flow is initiated by a main coordination layer, which uses NumPy's array partitioning tool np.array_split(df_raw,

num_cores) to divide the continuous 102,502-row matrix into multiple segments that are horizontally separated and independent in the host memory. When calling concurrent.futures.ProcessPoolExecutor, it sends a signal to the host operating system to execute independent fork() or spawn() subprocess commands, thereby instantiating isolated and independent work engine levels.

```
def multi_processing_loop_kernel(df_shard):
    prop_types, furnishing_list, prices_rm, prices_psf, built_up, land_area = [], [], [], [], [], []
    for idx, row in df_shard.iterrows():
        text = str(row['raw_text'])
        if re.search(r"Bungalow|Villa", text, re.IGNORECASE): prop_types.append("Bungalow/Villa")
        elif re.search(r"Condominium|Serviced Residence", text, re.IGNORECASE): prop_types.append("Condominium")
        elif re.search(r"Apartment|Flat", text, re.IGNORECASE): prop_types.append("Apartment/Flat")
        elif re.search(r"Terrace|Terraced|Link", text, re.IGNORECASE): prop_types.append("Terrace/Link")
        else: prop_types.append("Other")

        if re.search(r"Unfurnished", text, re.IGNORECASE): furnishing_list.append("Unfurnished")
        elif re.search(r"Partly|Partially", text, re.IGNORECASE): furnishing_list.append("Partially Furnished")
        elif re.search(r"Fully", text, re.IGNORECASE): furnishing_list.append("Fully Furnished")
        else: furnishing_list.append("Unknown")

        p_rm = re.search(r"RM\s*([\d,]+)", text)
        prices_rm.append(float(p_rm.group(1).replace(",", "")) if p_rm else np.nan)
        p_psf = re.search(r"RM\s*([\d,.]+)\s*psf", text, re.IGNORECASE)
        prices_psf.append(float(p_psf.group(1).replace(",", "")) if p_psf else np.nan)
        b_up = re.search(r"([\d,]+)\s*sqft(?:\s*\(floor\))?", text, re.IGNORECASE)
        built_up.append(float(b_up.group(1).replace(",", "")) if b_up else np.nan)
        l_area = re.search(r"([\d,]+)\s*sqft\s*\(land\)", text, re.IGNORECASE)
        land_area.append(float(l_area.group(1).replace(",", "")) if l_area else np.nan)

    df_shard['property_type'] = prop_types
    df_shard['furnishing'] = furnishing_list
    df_shard['price_rm'] = prices_rm
    df_shard['price_psf'] = prices_psf
    df_shard['built_up_sqft'] = built_up
    df_shard['land_area_sqft'] = land_area
    return df_shard
```

Figure 5.2 Architectural Mapping of Optimized Technique 1 (Multiprocessing)

Each independent subprocess is assigned to its own private and isolated virtual memory boundary and starts an independent Python interpreter runtime context. This approach eliminates the lock contention problem by transferring the execution to the true MIMD (Multi-Instruction, Multi-Data) state. However, the illustration reveals a key code-level performance bottleneck: within each independent process pool task, data operations still rely on row-by-row loops (such as for idx, row in df_shard.iterrows():). Therefore, although the pipeline can successfully map data processing to all available physical hardware channels, each sub-node is still affected by the high abstraction level and object allocation overhead specific to the high-level interpreter loop at the top level.

Figure 5.3 presents the execution performance of our highest optimization level - the Hybrid Data Engine. This architecture adopts a multi-layer design: through the distribution of

processes at the macro level, all physical computing cores are activated simultaneously, combined with vector calculations at the micro level, thereby eliminating internal row processing delays. In each sub-node workspace, the use of manual loop structures and row index iterations is completely prohibited. Instead, the script adopts operations aligned with vectors such as.loc[raw_text_series.str.contains(...)] to perform modification operations on the entire data block at once.

```python
def clean_dataframe_chunk(chunk_df):
    """
    Worker function executed concurrently across independent CPU cores.
    Combines core-level parallel processing with vectorized array assignments.
    """
    # Pre-allocate feature target columns instantly across the shard array
    chunk_df['price_rm'] = np.nan
    chunk_df['price_psf'] = np.nan
    chunk_df['built_up_sqft'] = np.nan
    chunk_df['land_area_sqft'] = np.nan
    chunk_df['property_type'] = ""
    chunk_df['furnishing'] = ""

    # Enforce strict string formatting across the target text block
    text_series = chunk_df['raw_text'].astype(str)

    # 1. Vectorized Category Mapping Rules (Bypasses row loops completely)
    chunk_df.loc[text_series.str.contains('Bungalow|Villa', na=False), 'property_type'] = 'Bungalow/Villa'
    chunk_df.loc[text_series.str.contains('Condominium|Serviced Residence', na=False), 'property_type'] = 'Condominium'
    chunk_df.loc[text_series.str.contains('Apartment|Flat', na=False), 'property_type'] = 'Apartment/Flat'
    chunk_df.loc[text_series.str.contains('Terrace|Terraced|Link', na=False), 'property_type'] = 'Terrace/Link'
    chunk_df.loc[text_series.str.contains('Semi-D|Semi-Detached', na=False), 'property_type'] = 'Semi-Detached'

    chunk_df.loc[text_series.str.contains('Unfurnished', na=False), 'furnishing'] = 'Unfurnished'
    chunk_df.loc[text_series.str.contains('Partly|Partially', na=False), 'furnishing'] = 'Partially Furnished'
    chunk_df.loc[text_series.str.contains('Fully', na=False), 'furnishing'] = 'Fully Furnished'

    # 2. Iterative Regular Expression Extractors (For complex non-uniform patterns)
    for index, row in chunk_df.iterrows():
        text = str(row['raw_text'])

        # Rule 1: Total Valuation Matrix Extraction
        price_match = re.search(r'RM\s*([\d,]+)', text)
        if price_match:
            chunk_df.at[index, 'price_rm'] = float(price_match.group(1).replace(',', ''))

        # Rule 2: Price Per Square Foot Density Evaluation
        psf_match = re.search(r'RM\s*([\d,.]+)\s*psf', text, flags=re.IGNORECASE)
        if psf_match:
            chunk_df.at[index, 'price_psf'] = float(psf_match.group(1).replace(',', ''))

        # Rule 3: Built-up Internal Area Calculations
        floor_match = re.search(r'([\d,]+)\s*sqft(?:\s*\(floor\))?', text, flags=re.IGNORECASE)
        if floor_match:
            chunk_df.at[index, 'built_up_sqft'] = float(floor_match.group(1).replace(',', ''))

        # Rule 4: Land Boundary Lot Dimensions Identification
        land_match = re.search(r'([\d,]+)\s*sqft\s*\(land\)', text, flags=re.IGNORECASE)
        if land_match:
            chunk_df.at[index, 'land_area_sqft'] = float(land_match.group(1).replace(',', ''))

    return chunk_df
```

Figure 5.3 High-Performance Multiprocessing Orchestrator

This layout maximizes performance by directly executing regular expression searches and character replacements .str.extract() in consecutive memory-mapped column blocks. Since the underlying string data is stored in a continuous raw data block form, the instructions utilize

pre-compiled native C extension functions, enabling the pipeline to use hardware-level SIMD (Single Instruction Multiple Data) execution paths. By performing calculations on large-scale array segments within one CPU clock cycle, the Hybrid Framework minimizes the number of interpreter call stack levels, completely avoids the delay of high-level object encapsulation, and achieves the highest data throughput.

6.0 Performance Evaluation

To systematically evaluate the engineering efficiency of our data processing workflow, a rigorous empirical performance test was conducted on a dataset of 102,502 original attribute records in a controlled hardware environment. To ensure statistical validity and eliminate temporary environmental anomalies such as dynamic operating system background thread context switching or CPU clock frequency adjustment, each architecture model underwent multiple independent and identical verification runs.

6.1 Before vs After Optimization

(A) Before Optimization

The baseline implementation processed the dataset sequentially using a row-by-row iteration approach based on iterrows(). During execution, each property record was processed individually, requiring repeated string parsing, regular expression matching, and feature extraction for every row. Since all operations were executed sequentially within a single Python interpreter, the workflow was unable to fully utilize the available multi-core processing capabilities.

In addition, the repeated creation of Python objects and frequent transitions between the Python interpreter and underlying C libraries introduced considerable computational overhead. As the dataset size increased to more than 100,000 records, the cumulative processing cost became increasingly significant, limiting the scalability and efficiency of the baseline implementation.

(B) **After Optimization**

To overcome the limitations of the sequential implementation, two optimization techniques were introduced.

The first optimization technique employed multiprocessing using ProcessPoolExecutor. The dataset was partitioned into multiple independent chunks, allowing each chunk to be processed concurrently by separate CPU processes. By distributing the workload across multiple processor cores, this approach was designed to improve parallel execution while avoiding the limitations imposed by Python's Global Interpreter Lock (GIL).

The second optimization technique adopted a hybrid vectorized approach, combining multiprocessing with Pandas vectorized operations. Instead of relying primarily on row-by-row processing, vectorized string operations and column-based computations were applied to process multiple records simultaneously. This reduced interpreter overhead while improving the efficiency of large-scale data manipulation.

The effectiveness of these optimization techniques was subsequently evaluated using three performance metrics: execution latency, peak memory allocation, and CPU core workload. The experimental results and comparative analysis are presented in the following section.

6.2 Comparison of Execution Latency, Peak Memory Allocation and CPU Core Workload

6.2.1 Overall Performance Comparison

This section compares the performance of the baseline implementation with the two proposed optimization techniques using three evaluation metrics: execution latency, peak memory allocation, and CPU core workload. These metrics were selected to evaluate both computational performance and resource utilization, providing a comprehensive assessment of each optimization approach.

Figure 6.1 compares the average execution latency of the baseline implementation and the two proposed optimization techniques. The Hybrid Vectorized approach achieved the shortest execution time, followed by the multiprocessing approach, while the baseline implementation recorded the highest latency, demonstrating the effectiveness of the optimization techniques.

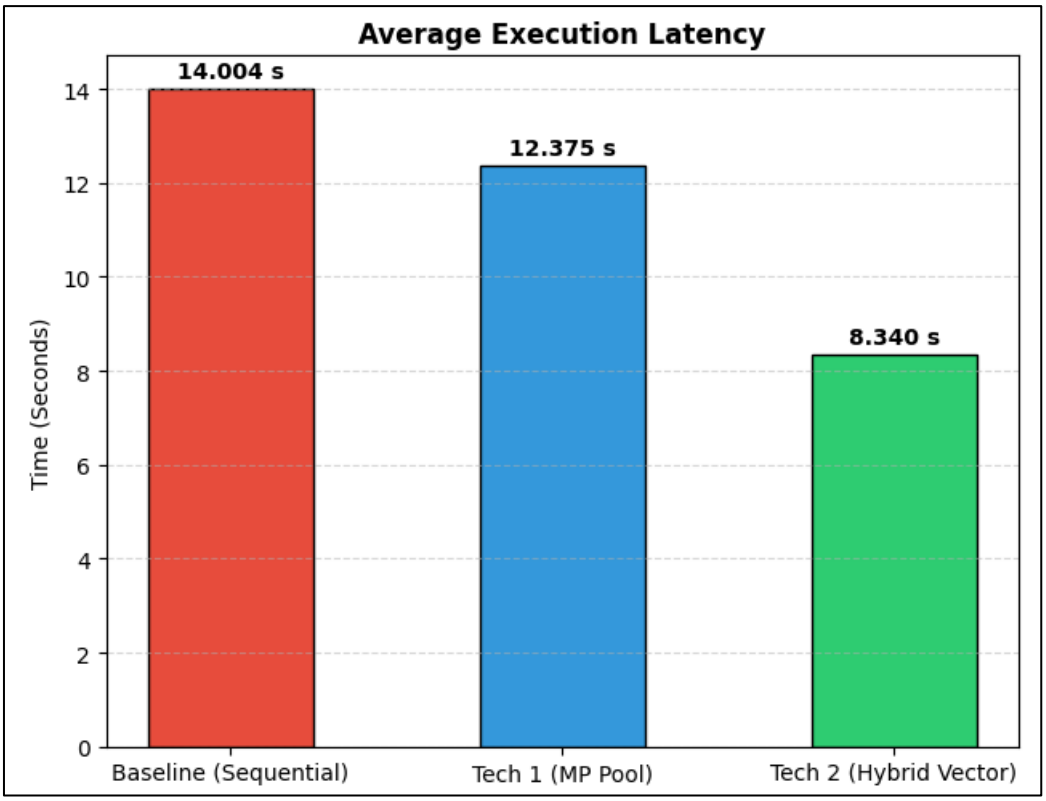


Figure 6.1 Bar Chart of Average Execution Latency between 3 Optimization Techniques

Figure 6.2 presents the average peak memory allocation for each implementation during data processing. Although the multiprocessing approach utilized the most memory due to the creation of multiple processes, the Hybrid Vectorized approach required the least memory, indicating more efficient resource utilization.

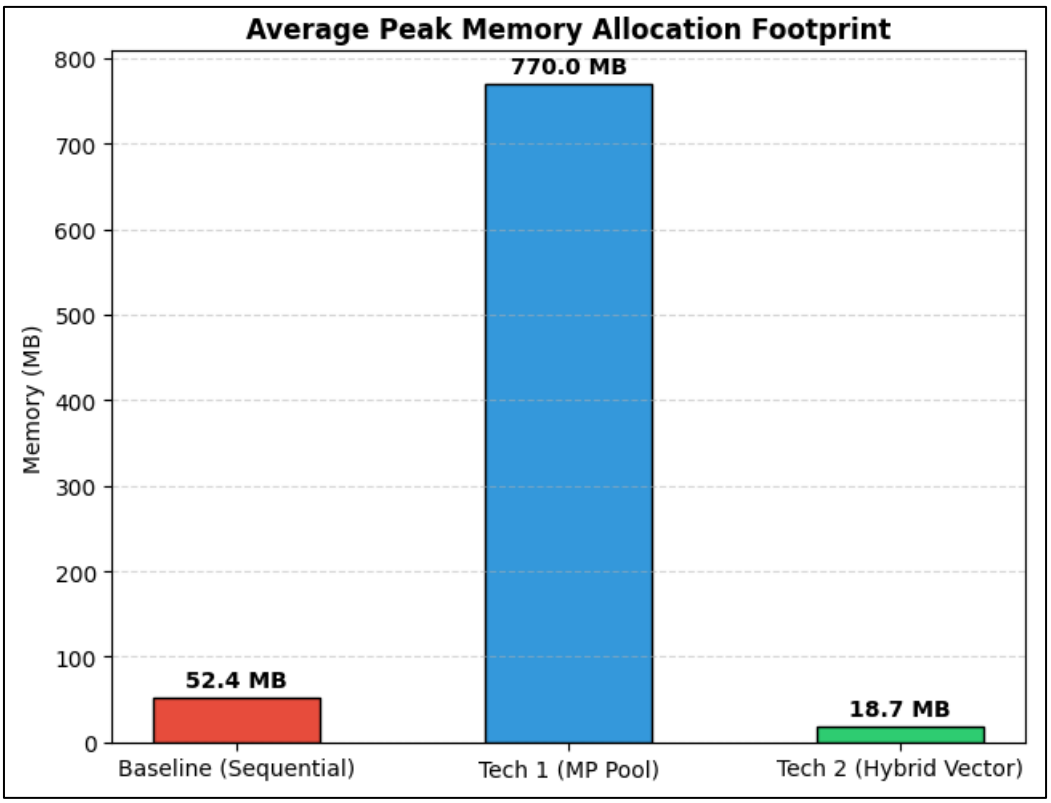


Figure 6.2 Bar Chart of Average Peak Memory Allocation Footprint between 3 Optimization Techniques

Figure 6.3 illustrates the average CPU core workload of the three implementations. The multiprocessing approach achieved the highest CPU utilization by distributing tasks across multiple cores, whereas the Hybrid Vectorized approach maintained moderate CPU usage while still delivering the best overall performance.

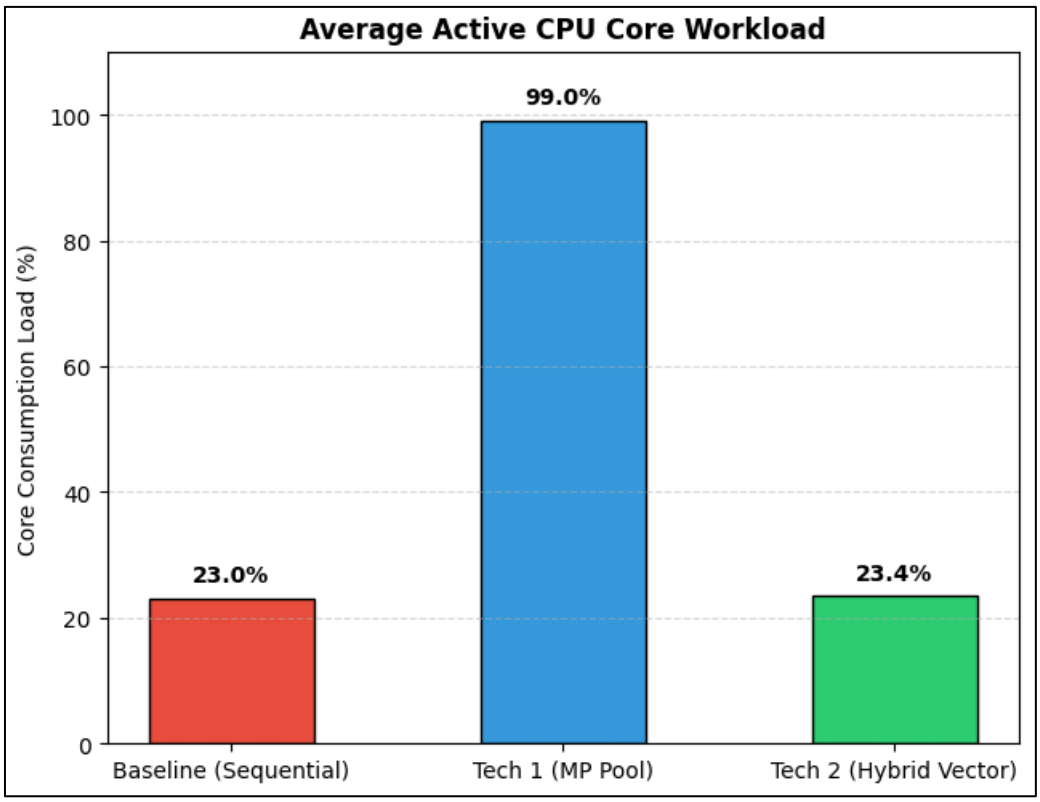


Figure 6.3 Bar Chart of Average Active CPU Core Workload between 3 Optimization Techniques

Table 6.1 shows that although Tech 1 (Multiprocessing Pool) achieved the highest average CPU core workload (99.0%), this primarily reflects its ability to utilize nearly all available CPU cores through parallel processing rather than indicating superior overall performance. The intensive use of multiple processes also resulted in substantially higher peak memory allocation (770.0 MB). In contrast, Tech 2 (Hybrid Vectorized) maintained a relatively low average CPU core workload (23.4%), comparable to the baseline implementation (23.0%), while simultaneously achieving the shortest execution latency (8.340 s) and the lowest peak memory allocation (18.7 MB). These results indicate that vectorized operations improve computational efficiency by reducing interpreter overhead and optimizing data processing without requiring intensive CPU utilization. Therefore, CPU workload should be interpreted

together with execution latency and memory allocation, as higher CPU utilization alone does not necessarily correspond to better overall performance.

Table 6.1 Analysis Comparative of Three Different Optimization Techniques

Metric	Baseline	Tech 1	Tech 2	Best Performer
Average Execution Latency (s)	14.004 s	12.375 s	**8.340 s**	Tech 2
Peak Memory Allocation (MB)	52.4 MB	770.0 MB	**18.7 MB**	Tech 2
Average Active CPU Core Workload (%)	23.0%	99.0%	23.4%	Depends on objective

*Baseline = Sequential, Technique 1 = Multiprocessing Pool, Technique 2 = Hybrid Vectorized

6.2.2 Overall Performance Analysis

Overall, as illustrated in Table 6.2, the experimental results indicate that Tech 2 (Hybrid Vectorized) provides the most effective and balanced optimization strategy among the three approaches. It achieved the shortest execution latency (8.340 s) while simultaneously recording the lowest peak memory allocation (18.7 MB) and maintaining moderate CPU utilization. These results demonstrate superior computational efficiency and resource management compared with both the baseline sequential implementation and the multiprocessing-based approach, making Tech 2 the most suitable solution for large-scale data processing tasks.

Table 6.2 Experimental Results of Three Different Optimization Techniques

Criteria	Baseline	Tech 1	Tech 2
Execution Speed	Slowest	Faster than baseline (≈11.6% improvement)	Fastest (≈40.4% improvement over baseline)
Memory Efficiency	Moderate	Highest memory consumption	Most memory-efficient
CPU Utilization	Low	Fully utilizes available CPU cores	Like baseline due to vectorized operations
Overall Performance	Baseline reference	Good parallel performance but expensive in memory	Best balance of speed and resource efficiency

*Baseline = Sequential, Technique 1 = Multiprocessing Pool, Technique 2 = Hybrid Vectorized

7.0 Challenges & Limitations

7.1 Challenges and Solutions

Table 7.1 Challenges, solutions and future improvements

Problems	Solutions	Future Improvements
HTTP 403 error using Requests crawler Large-scale crawling with Requests, BeautifulSoup, and ThreadPoolExecutor triggered HTTP 403 errors because the website restricted direct requests.	Switched to a Selenium-based crawler, which provided more stable browser-based access. The Requests crawler was retained only for testing.	Explore API-based extraction or a more efficient Requests crawler with rate limiting, retry logic, and compliance with website access policies.
Browser session instability Long Selenium runs occasionally produced invalid session errors when ChromeDriver crashed or disconnected.	Restarted Chrome, resumed crawling from the last successful page using the crawling log, and avoided restarting from page 1.	Implement automatic browser restart and session recovery to resume crawling without manual intervention.
Duplicate property listings The raw dataset contained 7,498 duplicate records because some listings appeared on multiple pages.	Preserved duplicates in the raw dataset and removed them later using listing_url during data cleaning.	Perform duplicate checking during crawling to prevent repeated records from being stored.
Unstructured raw listing text Property information was stored in a single raw_text field and could not be analyzed directly.	Preserved the original text and extracted structured attributes during the data processing stage.	Develop a dedicated parsing module using regular expressions and validation rules for more accurate extraction.

7.2 Limitations

(A) Dependence on Selenium

- The final crawler relied on Selenium for browser-based crawling.
- Selenium was more stable than the Requests-based crawler, but it was slower because each page had to be opened and loaded through Chrome before extraction.
- This increased the total crawling time when collecting more than 100,000 records.

(B) Requests-based crawler was not suitable for large-scale crawling

- A faster Requests + BeautifulSoup + ThreadPoolExecutor method was tested.
- It worked on early pages but later returned HTTP 403 access forbidden errors.
- Therefore, it could not be used as the final crawling solution.

(C) Local environment dependency

- The crawler depended on Windows CMD, Python virtual environment, Chrome, ChromeDriver, and local internet connection.
- If the browser crashed, internet disconnected, or laptop entered sleep mode, the crawler could stop unexpectedly.
- This made the crawling process dependent on local machine stability.

(D) Manual monitoring was required

- The crawler saved data and logs automatically, but it still required manual checking.
- The user had to monitor CMD output, restart the crawler after errors, clear Chrome processes, and update page ranges manually.
- Therefore, the system was semi-automated, not fully automated.

(E) Selenium session instability

- During long crawling sessions, Selenium sometimes produced invalid session id errors.
- This happened when Chrome or ChromeDriver disconnected during execution.
- When this occurred, the crawler had to be stopped and resumed from the last successful page using the crawling log.

8.0 Conclusion & Future Work

8.1 Summary of findings

This project successfully developed a large-scale web crawling and data processing pipeline for iProperty Malaysia. A total of 102,502 raw property listing records were collected and stored in CSV format. After duplicate removal based on listing URLs, the cleaned dataset contained 95,004 unique property records.

The data collection process used Selenium and BeautifulSoup to extract property listing data from paginated iProperty pages. Although a Requests-based crawler was tested for faster crawling, it returned HTTP 403 errors during larger-scale crawling. Therefore, Selenium was selected because it provided more stable browser-based crawling.

The data processing stage converted the raw listing text into a cleaner dataset. Duplicate records were removed, timestamps were converted, and regex patterns were used to extract important attributes such as price, price per square foot, built-up size, land area, property type, and furnishing status.

The optimization stage compared sequential processing, multiprocessing, and hybrid vectorized processing. The results showed that optimized methods improved throughput and CPU utilization compared with the sequential baseline. However, multiprocessing also introduced overhead from data splitting, process creation, and recombination.

Overall, this project showed how web crawling, data cleaning, and high-performance processing can be combined to handle large-scale property data. It also highlighted the importance of balancing speed, stability, data quality, and system resource usage.

8.2 Future Improvement

Although the hybrid multi-process and vectorized batch processing system achieves stable and highly scalable resource utilization, in future iteration designs, further improvement

of the original performance and resource usage efficiency can be achieved by optimizing multiple technical execution levels. The main direction of runtime acceleration lies in eliminating the row-by-row regular expression loops embedded within each sub-data fragment. The underlying text parsing and feature extraction logic can be recompiled using a low-level just-in-time (JIT) compiler, such as Numba, or directly integrated with the compiled native C++ extension. Moving the calculation of string patterns from the pure Python environment to the native machine instruction level will significantly reduce the number of CPU cycles at the element level. This change will completely bypass the bottleneck of the traditional interpreter engine, transforming the entire pipeline into a fully vectorized framework, enabling complex numerical extraction to be executed simultaneously and in parallel on memory units.

Furthermore, the architectural trade-offs that caused by the process isolation boundary provide significant opportunities for memory management optimization. To resolve the obvious serialization of processes and initialization overhead, the backend architecture can be reconfigured to adopt a zero-copy shared memory interface, combined with advanced memory storage technologies such as Apache Arrow, Plasma, or Ray. Under the current structural model, data arrays must be serialized as independent byte streams to navigate between different CPU core workspaces, resulting in local processing delays and significant memory duplication. After switching to a shared memory layout, child processes can safely access and modify different fragments of a single unified tracking block in the memory. This will completely eliminate the delay of data packaging, reducing the inter-process communication overhead to zero, and protecting the low-configured hardware infrastructure from execution failures caused by insufficient memory during data expansion cycles.

Finally, for enterprise-level large-scale operations, when data collection tasks involve millions of records, the engineering logic can be modified to migrate the task from the host CPU environment to a dedicated graphics processing unit (GPU) acceleration framework. Deploying modern GPU-based libraries, such as NVIDIA RAPIDS cuDF, enables string parsing, regular expression filtering, and structured feature mapping to be performed simultaneously on thousands of parallel CUDA and Tensor cores. This powerful single instruction, multiple data (SIMD) extension capability will eliminate the system's reliance on the original single-core clock frequency. When combined with an asynchronous dynamic load balancer, which can automatically distribute data shards based on text density rather than

simple row segmentation, these system optimizations will significantly improve the processing efficiency in large-scale data architectures.

9.0 References

Selenium. (2025). *The Selenium Browser Automation Project*. Selenium Documentation. https://www.selenium.dev/documentation/

Selenium. (2024). *WebDriver*. Selenium Documentation. https://www.selenium.dev/documentation/webdriver/

Richardson, L. (2026). *Beautiful Soup Documentation*. Crummy. https://www.crummy.com/software/BeautifulSoup/bs4/doc/

The pandas Development Team. (2026). *pandas Documentation*. pandas. https://pandas.pydata.org/docs/

The pandas Development Team. (2026). *pandas User Guide*. pandas. https://pandas.pydata.org/docs/user_guide/

Apache Software Foundation. (2025). *Apache Parquet Documentation*. Apache Parquet. https://parquet.apache.org/docs/

Apache Software Foundation. (2025). *Apache Parquet*. Apache Parquet. https://parquet.apache.org/

10.0 Appendices

Link to the repository

https://github.com/sean-seah/HPDP/tree/main/2526/project/BigProperty/project%201

11.0 Sample code snippets

Web Scraper

crawler.py

src > crawler.py > ...

```python
import time
import random
import csv
from datetime import datetime
from pathlib import Path

import pandas as pd
from bs4 import BeautifulSoup

from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

from webdriver_manager.chrome import ChromeDriverManager


BASE_URL = "https://www.iproperty.com.my/property-for-sale"
RAW_DATA_PATH = Path("data/raw/iproperty_raw_test.csv")
LOG_PATH = Path("data/logs/crawling_log.csv")


def start_driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--start-maximized")
    options.add_argument("--disable-blink-features=AutomationControlled")

    driver = webdriver.Chrome(
        service=Service(ChromeDriverManager().install()),
        options=options
    )
    return driver


def save_records(records):
    if not records:
        return

    df = pd.DataFrame(records)

    file_exists = RAW_DATA_PATH.exists()

    df.to_csv(
        RAW_DATA_PATH,
        mode="a",
        index=False,
        header=not file_exists,
        encoding="utf-8-sig"
    )
```

Web Scraper Source Code Part 1

```python
def save_log(page_number, status, records_collected, error_message=""):
    file_exists = LOG_PATH.exists()

    with open(LOG_PATH, mode="a", newline="", encoding="utf-8-sig") as file:
        writer = csv.writer(file)

        if not file_exists:
            writer.writerow([
                "page",
                "status",
                "records_collected",
                "time",
                "error"
            ])

        writer.writerow([
            page_number,
            status,
            records_collected,
            datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            error_message
        ])


def scrape_page(driver, page_number):
    url = f"{BASE_URL}?page={page_number}"
    print(f"\nOpening: {url}")

    driver.get(url)

    # wait for page body to load
    WebDriverWait(driver, 20).until(
        EC.presence_of_element_located((By.TAG_NAME, "body"))
    )

    time.sleep(random.uniform(4, 7))

    soup = BeautifulSoup(driver.page_source, "lxml")

    records = []
    seen_links = set()

    # collect all links first, then filter possible property listing links
    links = soup.find_all("a", href=True)

    for link in links:
        raw_text = " ".join(link.get_text(" ", strip=True).split())
        href = link.get("href", "")

        if not raw_text:
            continue

        # filter likely property listing cards
        is_property_text = "RM" in raw_text and ("sqft" in raw_text.lower() or "bed" in raw_text.lower())
        is_property_link = "/property/" in href or "/sale/" in href

        if is_property_text and is_property_link:
            if href in seen_links:
                continue
```

Web Scraper Source Code Part 2

```
            seen_links.add(href)

            if href.startswith("/"):
                full_url = "https://www.iproperty.com.my" + href
            else:
                full_url = href

            records.append({
                "page": page_number,
                "raw_text": raw_text,
                "listing_url": full_url,
                "scraped_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            })

    return records


Tabnine | Edit | Test | Explain | Document
def main():
    driver = start_driver()

    total_records = 0

    try:
        for page in range(5001, 5201):  # test page 2001 to 4000
            try:
                records = scrape_page(driver, page)
                save_records(records)
                save_log(page, "success", len(records))

                total_records += len(records)

                print(f"Page {page}: {len(records)} records collected")
                print(f"Total so far: {total_records}")

                time.sleep(random.uniform(3, 6))

            except Exception as e:
                print(f"Error on page {page}: {e}")
                save_log(page, "failed", 0, str(e))

    finally:
        driver.quit()

    print("\nCrawling completed.")
    print(f"Total records collected: {total_records}")
    print(f"Raw data saved to: {RAW_DATA_PATH}")
    print(f"Log saved to: {LOG_PATH}")


if __name__ == "__main__":
    main()
```

Web Scraper Source Code Part 3

12.0 Screenshots of output

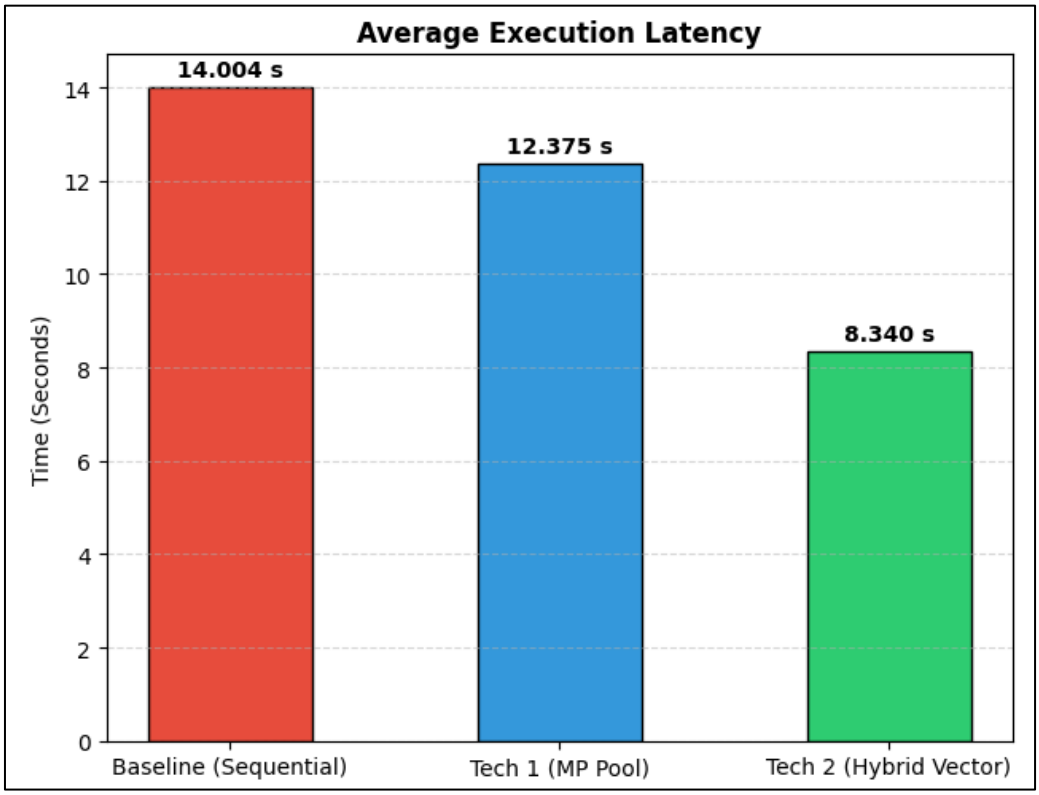


Figure 12.1 Bar Chart of Average Execution Latency between 3 Optimization Techniques

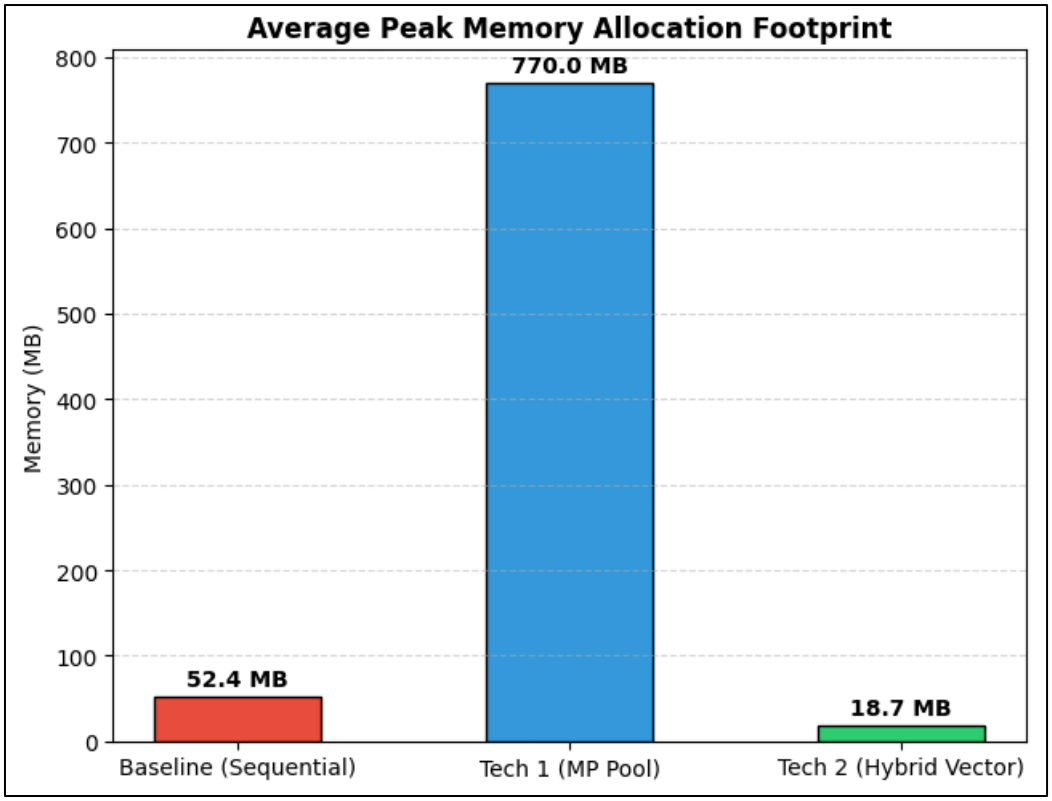


Figure 12.2 Bar Chart of Average Peak Memory Allocation Footprint between 3 Optimization Techniques

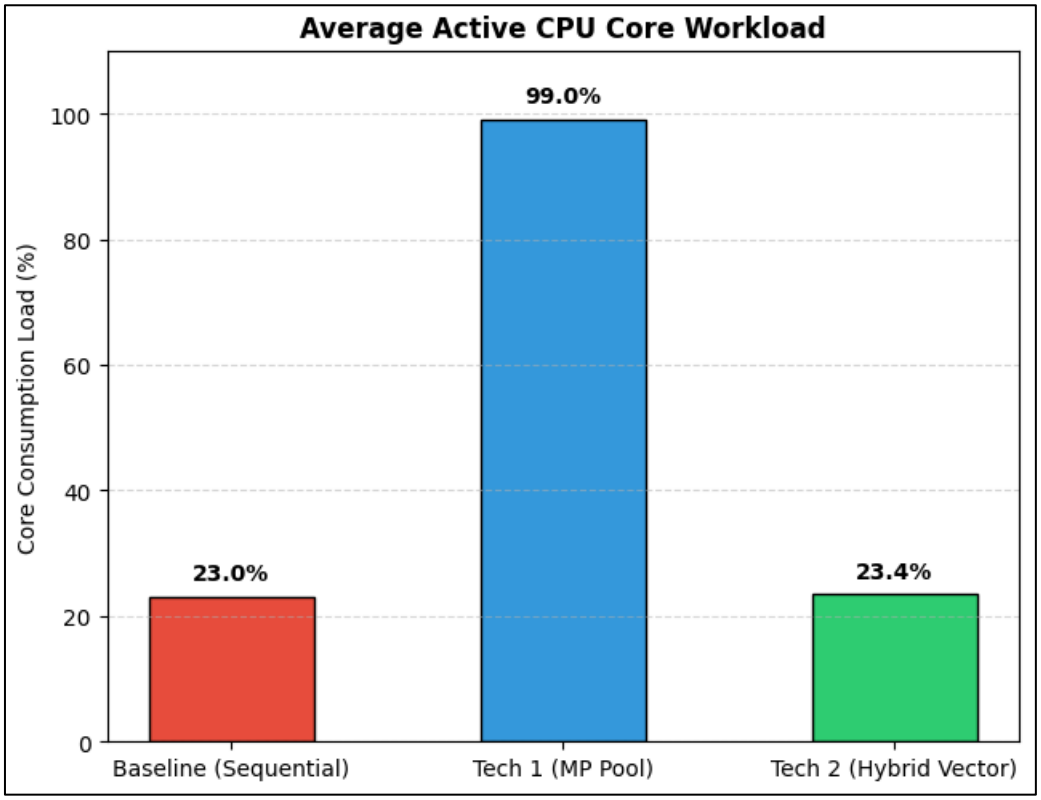


Figure 12.3 Bar Chart of Average Active CPU Core Workload between 3 Optimization Techniques